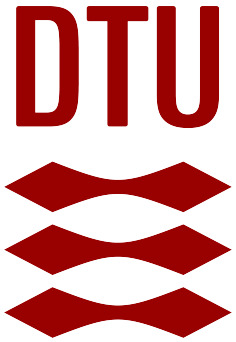

Board Game Report

Introduction to Artificial Intelligence

Course 02180

Authors

Michele Berlato - s250437

Marco Cola - s260201

Alexandros Kouros - s260329

Mattia Russo - s257370

1 Game rules

Kalaha (also known as Mancala or Kalah) is a deterministic two-player board game consisting of two rows of six pits and two larger reservoirs called Kalahs. The objective is straightforward: to accumulate more stones in your own Kalah than your opponent. On each turn, a player selects all stones from one of their pits and redistributes them one-by-one in a counter-clockwise direction, including their own Kalah but skipping the opponent's. The game's tactical depth arises from two key mechanics: Extra Turns, granted when the last stone lands in the player's Kalah, and Captures, triggered when the last stone lands in an empty pit on the player's side, allowing them to seize the stones in the opponent's opposite pit. The game concludes when one side of the board is cleared, at which point the player with the highest total count in their Kalah is declared the winner.

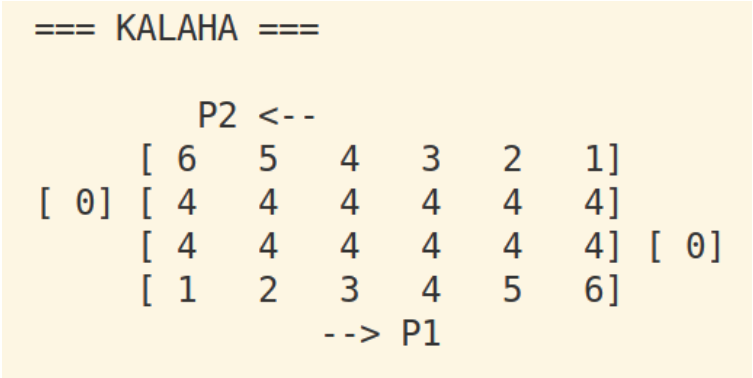

Figure 1: Initial board layout used in the implementation. Each player controls six pits and one kalah.

2 Game Type

The Kalaha game considered in this project has the following properties:

- **Players**: 2
- **Type**: Competitive
- **Payoff**: Zero-sum
- **Information**: Perfect information
- **Dynamics**: Deterministic
- **Turn structure**: Sequential turn-based
- **State observability**: Fully observable

Because Kalaha fits these criteria, it is highly susceptible to adversarial search algorithms. The most effective approach is the Minimax algorithm enhanced by Alpha-Beta pruning, which allows an AI to navigate the game tree efficiently by discarding sub-optimal branches that do not affect the final decision.

3 Size space

The state space of Kalaha, specifically the popular Kalah(6,4) variant is estimated to be between 10^{13} and 10^{15}. However, when considering the game tree complexity the total number of possible move sequences the figure rises significantly, often cited between 10^{18} and 10^{33}.

Approximate state space estimation

A rough upper bound for the state space of Kalaha can be obtained by observing that the game contains a fixed number of stones. In the Kalah(6,4) variant there are

$$48 = 12 \times 4$$

stones distributed across the board. These stones can occupy any of the 14 pits (12 regular pits and 2 stores). Ignoring game constraints, the number of ways to distribute 48 identical stones into 14 locations can be computed using the classical *stars and bars* combinatorial formula:

$$\binom{48 + 14 - 1}{14 - 1} = \binom{61}{13}$$

which corresponds to the number of integer solutions to

$$c_0 + c_1 + \cdots + c_{13} = 48$$

where c_i represents the number of stones in pit i. Evaluating this expression gives an upper bound of approximately

$$|S| \approx 4.7 \times 10^{11}$$

possible board configurations.
In practice the true state space is larger because the current player must also be considered as part of the state representation, giving approximately

$$|S| \approx 10^{12}$$

states.
This very large state space makes exhaustive search infeasible, motivating the use of depth-limited adversarial search algorithms combined with heuristic evaluation functions.
Specifically:

- **Depth-Limited Search:** Algorithms like Minimax must stop at a predefined depth to remain computationally feasible, using an evaluation function to estimate the value of non-terminal states.
- **Efficiency:** The use of Alpha-Beta pruning is mandatory to navigate this space, as it allows the AI to ignore vast branches of the tree that are mathematically proven to be sub-optimal, focusing its resources on the most promising lines of play.

In a typical game state a player can choose between at most six pits, giving an approximate branching factor of $b \approx 6$. If the search depth is d, the theoretical complexity of Minimax search is therefore

$$O(b^d)$$

which quickly becomes infeasible for large depths. For example, with $b = 6$ and $d = 8$, the number of explored nodes would already exceed 1.6×10^6. This motivates the use of Alpha-Beta pruning, which can reduce the effective branching factor and allow deeper search.

4 Elements of the game

Kalaha can be formally defined as a search problem

$$G = (s_0, Player(s), Action(s), Result(s, a), Terminal - Test(s), Utility(s, p))$$

where each component is defined as follows:

- **Initial state s_0:** A board with 12 pits, each containing 4 stones, and 2 empty Kalahs.

- **Player(s):** Returns the player whose turn it is, the player may move multiple times if the "extra turn" rule is triggered.
- **Action(s):** Selecting any non-empty pit on the current player's side of the board.
- **Result(s,a):** The state resulting from "sowing" the stones from the chosen pit counter-clockwise, including captures and extra turn logic.
- **Terminal-Test(s):** True if all pits on either player's side are empty.
- **Utility(s,p):** Calculated at the terminal state as the difference between stones in the player's Kalah and the opponent's.

$$Utility(s, p) = Store_p(s) - Store_{opp}(s)$$

5 AI states and moves representation

In the implementation the board state is represented as a vector of length 14:

$$s = (c_0, c_1, \ldots, c_{13})$$

where each c_i represents the number of stones contained in pit i. Indices $0 - 5$ correspond to Player 1 pits, index 6 to Player 1's store, indices $7 - 12$ to Player 2 pits and index 13 to Player 2's store.
Moves are represented as the index of the pit from which the "sowing" process begins. Since the rules for stone distribution, captures, and extra turns are deterministic, the transition from one state to the next requires no additional information beyond the chosen index.
It's not necessary to represent game states as belief states, because Kalaha is a game of full observability, the current "physical" position of the stones is the only information needed.

6 Methods & Algorithms

For the development of a Kalaha AI, the most appropriate choice among the methods discussed in the course is the Minimax algorithm enhanced with Alpha-Beta pruning.
This selection is justified by the game's nature: as a deterministic, zero-sum game of perfect information, Minimax can theoretically map every possible evolution of the match to find the optimal strategy while the alpha beta pruning allow to ignore branches that are mathematically proven to be worse than previously explored options, allowing to doubles the searchable depth.
The value of a state in the *Minimax* framework is defined recursively as

$$V(s) = \begin{cases} Utility(s) & \text{if } Terminal - Test(s) \\ \max_{a \in Actions(s)} V(Result(s, a)) & \text{if the current player is MAX} \\ \min_{a \in Actions(s)} V(Result(s, a)) & \text{if the current player is MIN} \end{cases}$$

In practice the search tree must be truncated at a finite depth d. In that case a heuristic evaluation function $h(s)$ is used instead of the true utility value.
Alpha-Beta pruning improves the efficiency of Minimax by maintaining two bounds:

α = best value found so far for MAX

β = best value found so far for MIN

Whenever the condition

$$\alpha \geq \beta$$

is satisfied, the remaining branches of the search tree can be pruned because they cannot influence the final decision.

Concrete Example of Algorithm Operation

To illustrate the operation of the Minimax algorithm with Alpha-Beta pruning within the context of Kalaha, consider a simplified end-game scenario with a search depth of d=2:

1. **Initial State s_0:** It is the turn of MAX (Player 1). Suppose they have two legal moves: emptying pit 4 (Move A) or pit 5 (Move B).
2. **Level 1 (Expansion):** The algorithm generates two child nodes, $n1$ (the result of Move A) and $n2$ (the result of Move B).
3. **Level 2 (MIN Evaluation):**For each of MAX's moves, the opponent (MIN) evaluates their optimal responses:
   - In branch $n1$, MIN has two responses leading to heuristic values h=5 and h=3. MIN will choose the minimum (3). The backup value for the root node becomes $\alpha = 3$.
   - In branch $n2$, the algorithm evaluates MIN's first response, which returns h=2.
4. **Pruning:**Since the value found (2) is already lower than the best-guaranteed value for MAX ($\alpha = 3$), the pruning condition is satisfied:
   $$\alpha \geq \beta \implies 3 \geq 2$$
   The algorithm therefore ignores the remaining sub-branches of n_2, as they cannot influence the final utility $V(s)$.

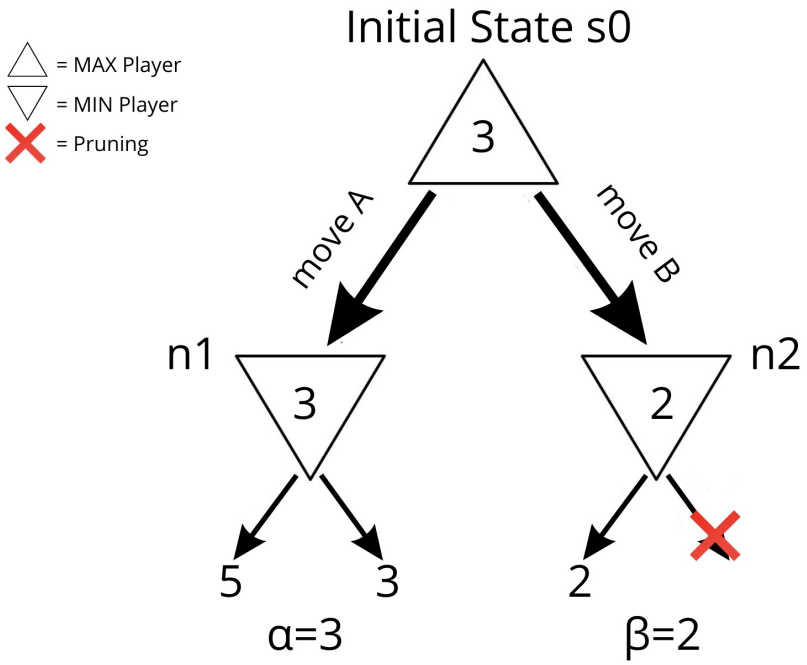

Figure 2: A simplified search tree illustrating Alpha-Beta pruning starting from s_0. The algorithm explores the left branch first, finding $\alpha = 3$. Upon evaluating the first node of the right branch ($\beta = 2$), the condition $\alpha \geq \beta$ is met, and the remaining sub-branch is pruned.

7 Heuristics and Evaluation functions

For non-terminal states, the AI player utilizes an advanced heuristic evaluation function $h(s, p)$. This function estimates the positional advantage of player p by considering a weighted combination of material count and strategic bonuses:

$$h(s,p) = 20 \cdot \Delta Store + \Delta Pits + Bonus_{extra} + Bonus_{capture}$$

Where:

- $\Delta Store$: The difference between the seeds in player p's store ($Store_p$) and the opponent's store ($Store_{opp}$). This is the primary objective of the game.
- $\Delta Pits$: The difference between the total seeds remaining in p's pits and the opponent's pits. This serves as a measure of potential future points and defensive stability.
- $Bonus_{extra}$: A strategic reward (set to 15) added if the move leads to an extra turn. This encourages the AI to prioritize move chains that maintain board control.
- $Bonus_{capture}$: A tactical reward (set to 10) added if the move results in a capture.

By assigning a high weight (20) to the store difference, the AI prioritizes scoring above all else. However, the inclusion of the pit difference and dynamic bonuses prevents "myopic" behavior. The AI will avoid moves that score a single point but leave its side vulnerable, and it will actively seek out tactical opportunities like captures and extra turns that a simpler heuristic would overlook.

8 Implemented AI's parameters

The implemented AI player allows several parameters to be adjusted:

- **Search depth (d):** determines how many moves ahead the algorithm explores the game tree.
- **Search algorithm:** either standard Minimax or Minimax with Alpha-Beta pruning.

In the implementation the default search depth is $d = 7$. Increasing the depth generally improves playing strength but significantly increases computational cost.
The program also allows AI vs AI matches to benchmark different parameter settings and compare the performance of Minimax and Alpha-Beta pruning.

Benchmark Results

We compared both the computational performance and playing strength of Minimax and Alpha-Beta pruning across multiple search depths. Each configuration was evaluated over $N = 200$ simulated games. To eliminate first-player bias, experiments were conducted twice with swapped player roles, and the results were averaged.
The main parameters varied in the experiments are the search depth d and the choice of search algorithm (Minimax vs Alpha-Beta pruning).

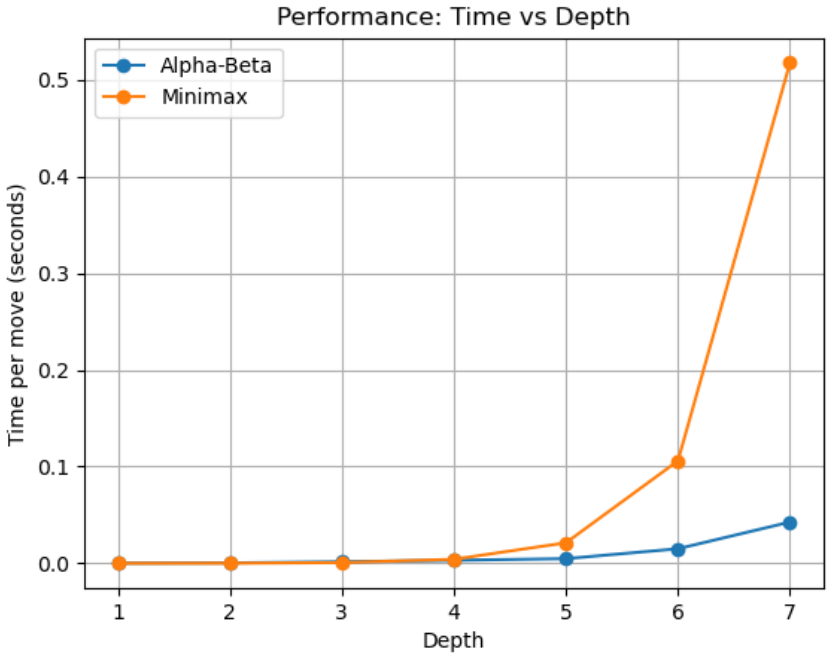

Figure 3: Average time per move as a function of search depth for Minimax and Alpha-Beta pruning.

As shown in Figure 3, Alpha-Beta pruning consistently requires significantly less time per move than standard Minimax at all tested depths. The difference becomes especially pronounced at higher depths, where Minimax exhibits rapid exponential growth in computation time. At depth 7, Minimax becomes substantially slower, while Alpha-Beta remains computationally feasible.

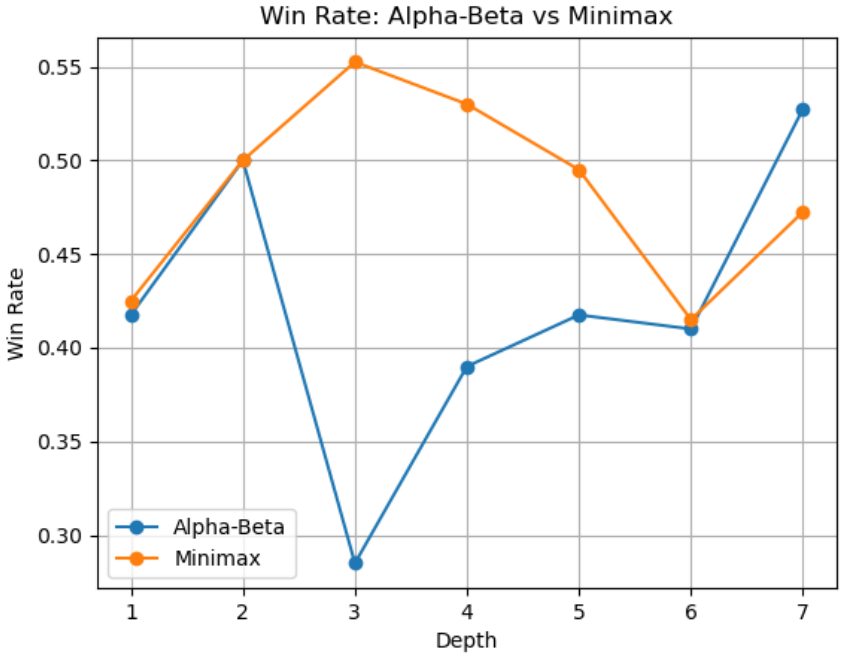

Figure 4: Win rate comparison between Alpha-Beta and Minimax across different depths. Results are averaged over both player roles to eliminate first-player bias.

Figure 4 shows that the win rates of Minimax and Alpha-Beta pruning are overall very similar across all tested depths. This is expected, as Alpha-Beta pruning does not alter the decision-making process of

Minimax, but only improves its efficiency by pruning branches that cannot influence the final outcome. Minor differences in win rate are due to stochastic elements in the experimental setup and the finite number of simulated games.

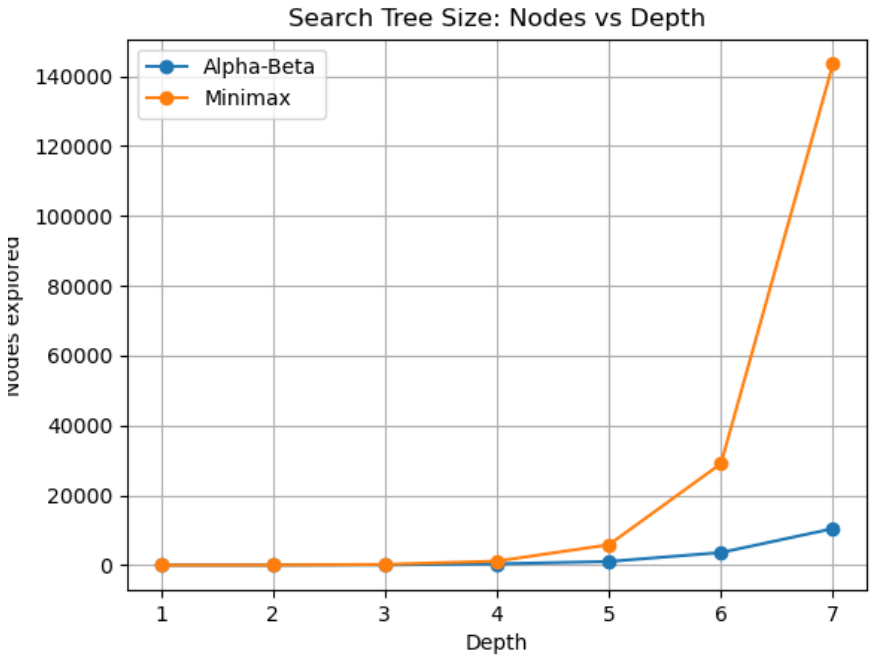

Figure 5: Number of nodes explored by Minimax and Alpha-Beta pruning as a function of search depth.

Figure 5 shows the number of nodes explored during the search for both algorithms. As expected, Minimax exhibits exponential growth in the size of the search tree as the depth increases. In contrast, Alpha-Beta pruning significantly reduces the number of explored nodes by eliminating branches that cannot influence the final decision. At higher depths, the reduction becomes substantial, explaining the performance improvements observed in Figure 3.
These results align with the theoretical complexity of adversarial search. While Minimax has a time complexity of $O(b^d)$, Alpha-Beta pruning reduces the effective branching factor by eliminating suboptimal branches, allowing deeper exploration within practical time limits.

9 Future work

Several improvements could further enhance the AI performance:

- **Transposition tables:** storing previously evaluated states to avoid redundant computations.
- **Iterative deepening search:** combining depth-limited search with time constraints.
- **Monte Carlo Tree Search:** an alternative algorithmic approach.